

Lib Func

标准库

C 标准库

C 标准库操作字符数组 `char[]/const char*`。

参见：[fprintf](#)、[fscanf](#)、[空终止字节字符串](#)

- `printf("%s", s)`：用 `%s` 来输出一个字符串（字符数组）。
- `scanf("%s", &s)`：用 `%s` 来读入一个字符串（字符数组）。
- `sscanf(const char *__source, const char *__format, ...)`：从字符串 `__source` 里读取变量，比如 `sscanf(str, "%d", &a)`。
- `sprintf(char *__stream, const char *__format, ...)`：将 `__format` 字符串里的内容输出到 `__stream` 中，比如 `sprintf(str, "%d", i)`。
- `strlen(const char *str)`：返回从 `str[0]` 开始直到 `'\0'` 的字符数。注意，未开启 O2 优化时，该操作写在循环条件中复杂度是 $O(n)$ 的。
- `strcmp(const char *str1, const char *str2)`：按照字典序比较 `str1` `str2` 若 `str1` 字典序小返回负值，两者一样返回 0，`str1` 字典序更大则返回正值。请注意，不要简单的认为返回值只有 0、1、-1 三种，在不同平台下的返回值都遵循正负，但并非都是 0、1、-1。
- `strcpy(char *str, const char *src)`：把 `src` 中的字符复制到 `str` 中，`str` `src` 均为字符数组头指针，返回值为 `str` 包含空终止符号 `'\0'`。
- `strncpy(char *str, const char *src, int cnt)`：复制至多 `cnt` 个字符到 `str` 中，若 `src` 终止而数量未达 `cnt` 则写入空字符到 `str` 直至写入总共 `cnt` 个字符。
- `strcat(char *str1, const char *str2)`：将 `str2` 接到 `str1` 的结尾，用 `*str2` 替换 `str1` 末尾的 `'\0'` 返回 `str1`。
- `strstr(char *str1, const char *str2)`：若 `str2` 是 `str1` 的子串，则返回 `str2` 在 `str1` 的首次出现的地址；如果 `str2` 不是 `str1` 的子串，则返回 `NULL`。
- `strchr(const char *str, int c)`：找到在字符串 `str` 中第一次出现字符 `c` 的位置，并返回这个位置的地址。如果未找到该字符则返回 `NULL`。
- `strrchr(const char *str, int c)`：找到在字符串 `str` 中最后一次出现字符 `c` 的位置，并返回这个位置的地址。如果未找到该字符则返回 `NULL`。

C++ 标准库

C++ 标准库操作字符串对象 [std::string](#)，同时也提供对字符数组的兼容。

参见：[std::basic_string](#)、[std::basic_string_view](#)

- 重载了赋值运算符 `+`，当 `+` 两边是 `string/char/char[]/const char*` 类型时，可以将这两个变量连接，返回连接后的字符串（`string`）。
- 赋值运算符 `=` 右侧可以是 `const string/string/const char*/char*`。
- 访问运算符 `[cur]` 返回 `cur` 位置的引用。
- 访问函数 `data()/c_str()` 返回一个 `const char*` 指针，内容与该 `string` 相同。
- 容量函数 `size()` 返回字符串字符个数。
- `find(ch, start = 0)` 查找并返回从 `start` 开始的字符 `ch` 的位置；`rfind(ch)` 从末尾开始，查找并返回第一个找到的字符 `ch` 的位置（皆从 0 开始）（如果查找不到，返回 -1）。
- `substr(start, len)` 可以从字符串的 `start`（从 0 开始）截取一个长度为 `len` 的字符串（缺省 `len` 时代码截取到字符串末尾）。
- `append(s)` 将 `s` 添加到字符串末尾。

- `append(s, pos, n)` 将字符串 `s` 中, 从 `pos` 开始的 `n` 个字符连接到当前字符串结尾。
 - `replace(pos, n, s)` 删除从 `pos` 开始的 `n` 个字符, 然后在 `pos` 处插入串 `s`。
 - `erase(pos, n)` 删除从 `pos` 开始的 `n` 个字符。
 - `insert(pos, s)` 在 `pos` 位置插入字符串 `s`。
 - `std::string` 重载了比较逻辑运算符, 复杂度是 $O(n)$ 的。
-

本页面最近更新: 2024/4/5 13:44:21, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Frankaiyou](#), [henryttrue](#), [StudyingFather](#), [zymooll](#), [Chrogeek](#), [Enter-tainer](#), [flyhahe](#), [lr1d](#), [ksyx](#), [NachtgeistW](#), [shuzhouliu](#), [Tiphereth-A](#), [tomirillaris](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用